



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Samuel Lebrero
02/12/2025



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
- Summary of all results

Introduction

- Project background and context
- Problems you want to find answers

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
 - Describe how data was collected
- Perform data wrangling
 - Describe how data was processed
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - How to build, tune, evaluate classification models

Data Collection

- For this project the data was collected via:
 - **SpaceX API.** A RESTful API which allowed to obtain data using a get request in conjunction with some auxiliary functions that performed a series of request to the SpaceX API in order to extract more specific data.
 - **Web Scraping.** This technique was employed on a Wikipedia page containing records of Falcon 9 historical launch ([*List of Falcon 9 and Falcon Heavy launches*](#))
- This data went through an initial data wrangling process, replacing some missing data and transforming it into a proper dataframe to be used later.

Data Collection – SpaceX API

- The response containing the data obtained by the request to the SpaceX API is parsed into JSON format. Afterward it's then converted into a Pandas dataframe.
- Using the aforementioned auxiliary functions on the existing dataframe, a series of requests to the SpaceX API are performed to create a dictionary.
- That dictionary is used to form a dataframe that after some initial preprocessing could be used for our purposes.
- https://github.com/slebra00/Coursera_DataScience/blob/Oa0398f6f7adbf066f946193aec0c6ef2c2a43ab/CapstoneProject/1_jupyter-labs-spacex-data-collection-api.ipynb

```
static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/188-050321EN-SkillsNetwork/datasets/API_call_spacex_api.json'
response=requests.get(static_json_url)
```

```
# Use json_normalize method to convert the json result into a dataframe
data=pd.json_normalize(response.json())
```

```
# Call getBoosterVersion
getBoosterVersion(data)
```

```
# Call getLaunchSite
getLaunchSite(data)
```

```
# Call getPayloadData
getPayloadData(data)
```

```
# Call getCoreData
getCoreData(data)
```

Finally lets construct our dataset using the data we have obtained. We we combine the columns into a dictionary.

```
launch_dict = {'FlightNumber': list(data['flight_number']),
               'Date': list(data['date']),
               'BoosterVersion': BoosterVersion,
               'PayloadMass': PayloadMass,
               'Orbit': Orbit,
               'LaunchSite': LaunchSite,
               'Outcome': Outcome,
               'Flights': Flights,
               'GridFins': GridFins,
               'Reused': Reused,
               'Legs': Legs,
               'LandingPad': LandingPad,
               'Block': Block,
               'ReusedCount': ReusedCount,
               'Serial': Serial,
               'Longitude': Longitude,
               'Latitude': Latitude}
```

Then, we need to create a Pandas data frame from the dictionary launch_dict.

```
# Create a data from launch_dict
data = pd.DataFrame.from_dict(launch_dict)
```


Data Collection - Scraping

- The request response is processed using the BeautifulSoup library, that allow us to locate and work with the table containing relevant information for our case.
- The table is then dissected into its columns and the information is extracted to different lists that form a dictionary.
- That dictionary is then used to create a Pandas dataframe.
- https://github.com/slebra00/Coursera_DataScience/blob/0a0398f6f7adbf066f946193aec0c6ef2c2a43ab/CapstoneProject/2_jupyter-labs-webscraping.ipynb

```
# use requests.get() method with the provided static_url and headers
# assign the response to a object
response = requests.get(static_url, headers=headers)

# Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(response.text, 'html.parser')

# Use the find_all function in the BeautifulSoup object, with element type 'table'.
# Assign the result to a list called 'html_tables'
html_tables = soup.find_all('table')

column_names = []

# Apply find_all() function with 'th' element on first_launch_table
# Iterate each th element and apply the provided extract_column_from_header() to get a column name
# Append the Non-empty column name ('if name is not None and len(name) > 0') into a list called column_names

for row in first_launch_table.find_all('th'):
    name = extract_column_from_header(row)
    if name is not None and len(name) > 0:
        column_names.append(name)

launch_dict=dict.fromkeys(column_names)

# Remove an irrelevant column
del launch_dict['Date and time ( )']

# Let's initial the launch_dict with each value to be an empty list
launch_dict['Flight No.']= []
launch_dict['Launch site']= []
launch_dict['Payload']= []
launch_dict['Payload mass']= []
launch_dict['Orbit']= []
launch_dict['Customer']= []
launch_dict['Launch outcome']= []
# Added some new columns
launch_dict['Version Booster']=[]
launch_dict['Booster Landing']=[]
launch_dict['Date']=[]
launch_dict['Time']=[]

.....Extract each table.....

df= pd.DataFrame([_key:pd.Series(value) for _key, value in launch_dict.items()])
df.head()
```

Data Wrangling

- The collected dataframe has been filtered to only contain entries corresponding to Falcon 9 launches (*BoosterVersion* column).
- The missing values in *PayloadMass* were addressed during the creation of the dataframe and were replaced by the mean value.
- The target variable selected (*Outcome*) is encoded into a binary label for future classification under the column name 'Class' with values:
 - 1 for all successful entries
 - 0 for everything else.
- https://github.com/slebra00/Coursera_DataScience/blob/0a0398f6f7adbf066f946193aec0c6ef2c2a43ab/CapstoneProject/3_labs-jupyter-spacex-Data%20wrangling.ipynb

EDA with Data Visualization

- During the EDA a number of charts were charted employing the libraries Plotly and Seaborn in order to gain some insights into the relationship between variables in the data, and how they relate to our target variable.
- The most employed type of chart was the Scatter point, which allowed to visualize the relation between 2 variables and how it's distributed against one another. Moreover, it also can color out target class in there as well.
- Also, a Bar plots was used for comparing categorical data; and a Line chart to visualize the trend of the target variable.
- https://github.com/slebra00/Coursera_DataScience/blob/Oa0398f6f7adbf066f946193aec0c6ef2c2a43ab/CapstoneProject/5_edadataviz.ipynb

EDA with SQL

- The purpose of the SQL queries performed were:
 - Identify how many distinct elements are in some fields like *LaunchSite*
 - Obtain maximum, minimum, total and average values for some numerical columns like *PayloadMass*
 - Ensure that the data only contains Falcon 9 data.
 - Gain an insight in the distribution and number of entries in relation to key fields like *Mission_Outcome* or *Landing_Outcome*, which are what we aim to predict.
- https://github.com/slebra00/Coursera_DataScience/blob/0a0398f6f7adb066f946193aec0c6ef2c2a43ab/CapstoneProject/4_jupyter-labs-eda-sql-coursera_sqlite.ipynb

Build an Interactive Map with Folium

- A Folium world map was created to contain circles showing the location of the different launch sites for the Falcon 9 entries.
- In addition, a number of markers equal to the number of entries for each location are created at those locations. Those markers are colored in green for successful cases and red for failures.
- This way we can easily glean insights of the Falcon 9 data related to the location of the launches.
- https://github.com/slebra00/Coursera_DataScience/blob/Oa0398f6f7adbf066f946193aec0c6ef2c2a43ab/CapstoneProject/6_lab_jupyter_launch_site_location.ipynb

Build a Dashboard with Plotly Dash

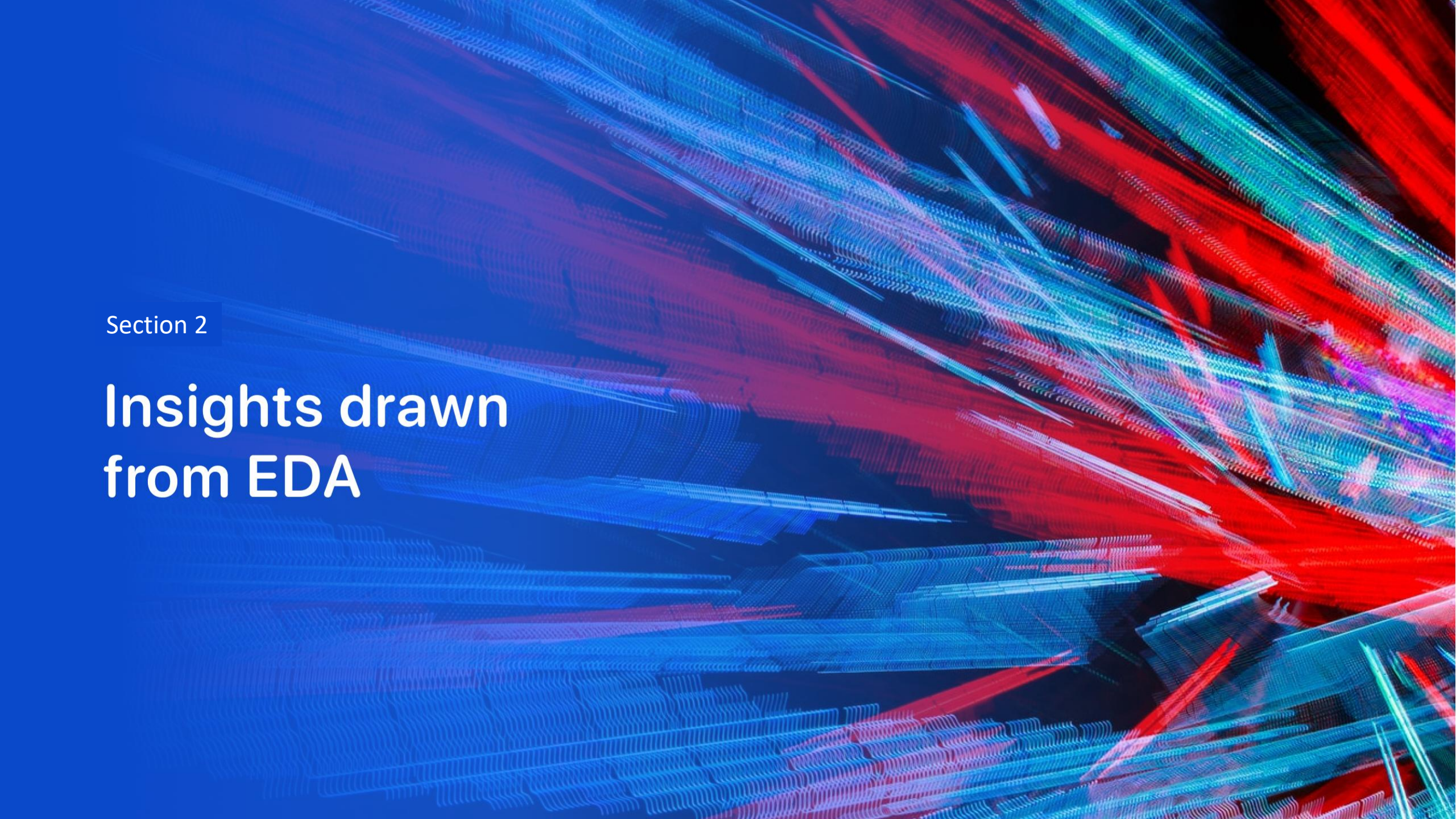
- For the dashboard we have implemented:
 - A Launched Site drop-down component. That allow to filter the data to be employed in the graphs, showing either the data for all sites at the same time or only data related to one specific site.
 - A Pie chart. That shows the ratio between successes and failures for the selected site.
 - A Range slider. Allowing to select between which Payload values, the data for the scatter plot is rendered.
 - A Scatter chart, that plots the success class (*y axis*) against the Payload mass (*x axis*)
- https://github.com/slebra00/Coursera_DataScience/blob/Oa0398f6f7adb066f946193aec0c6ef2c2a43ab/CapstoneProject/7_spacex-dash-app.py

Predictive Analysis (Classification)

- First the dataframe is divided into training and test sets, performing a few simple transformations and using the Scikit-learn library to create those sets.
 - The training set will contain 80% of the examples in the dataframe, with the other 20% used for evaluating the different trained models.
- We test various models: SVM, DecisionTrees, K-NN and LogisticRegression; to find the best performing one.
 - In order to try to ensure the best possible results for each model, GridSearchCV is employed to tune the hyperparameters.
- Finally, the model is evaluated against the test set.
- https://github.com/slebra00/Coursera_DataScience/blob/Oa0398f6f7adbf066f946193aec0c6ef2c2a43ab/CapstoneProject/8_SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb

Results

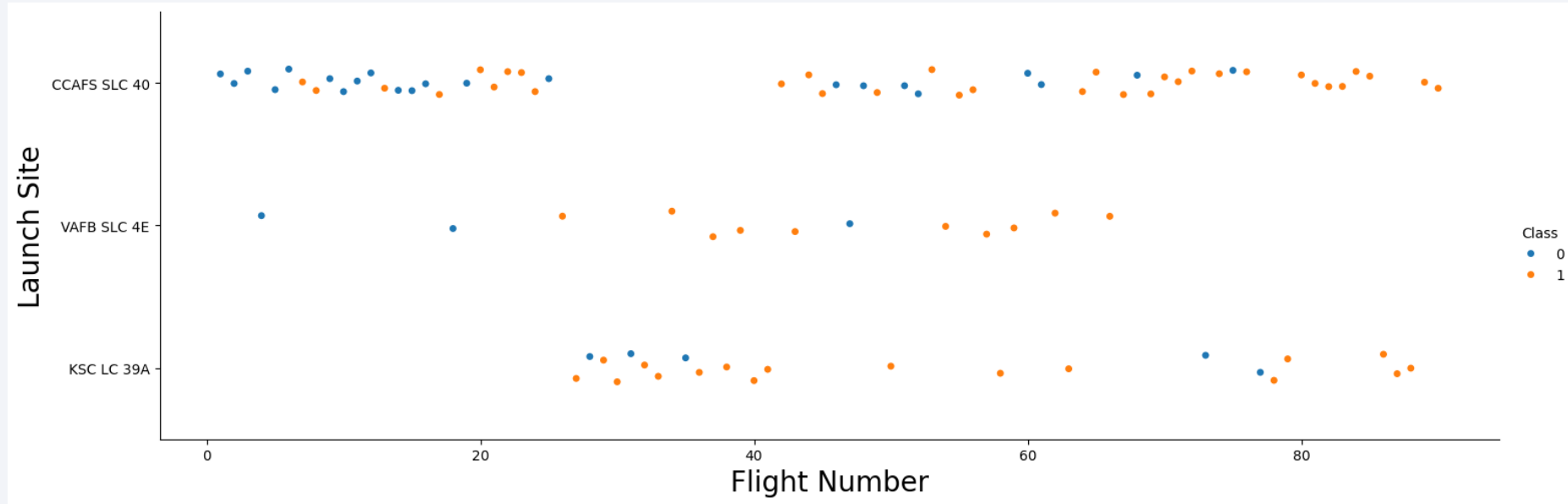
- With the exploratory data analysis it's possible to observe some correlation between the payload mass and the type of orbit towards the success. As well, as a clear upward trend in the success rate over the years.
- Interactive analytics demo in screenshots
- Predictive analysis results

The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of red and cyan. A faint, light blue grid pattern is also visible, particularly in the lower half of the image. The overall effect is dynamic and technological.

Section 2

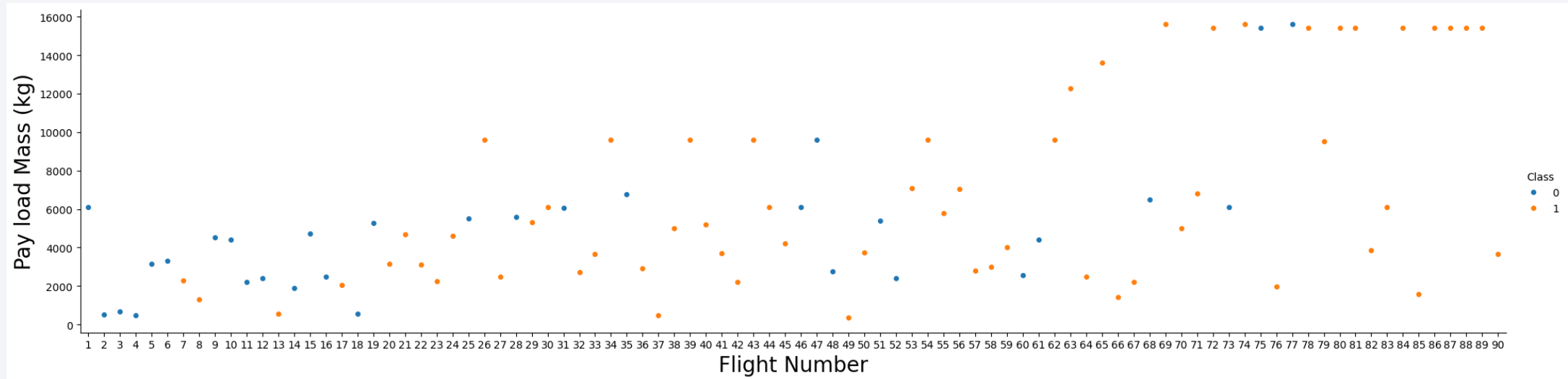
Insights drawn from EDA

Flight Number vs. Launch Site



- We this chart we see that as the number of flights increases the success rate does the same. And that the site KSC LC 39A don't have data of examples with low flight numbers.

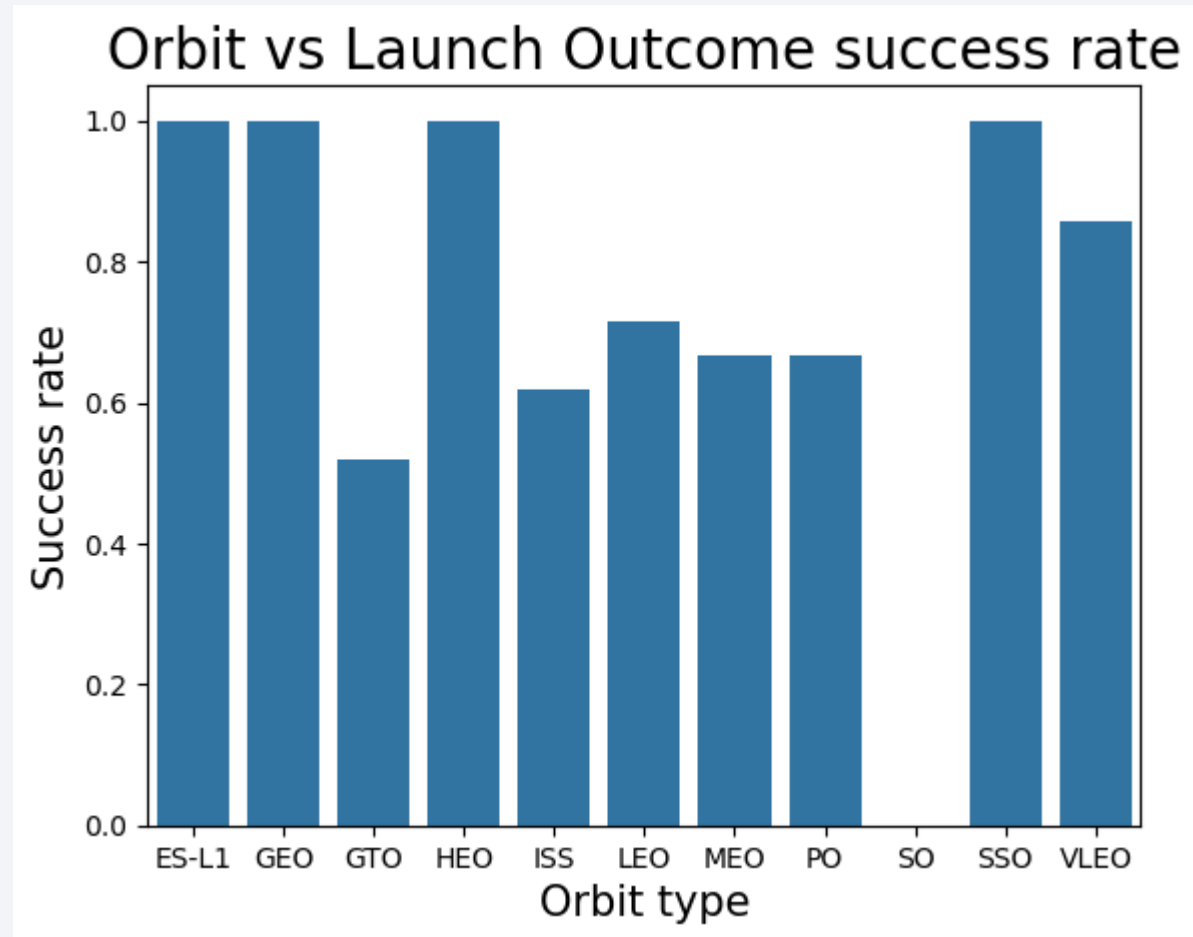
Payload vs. Launch Site



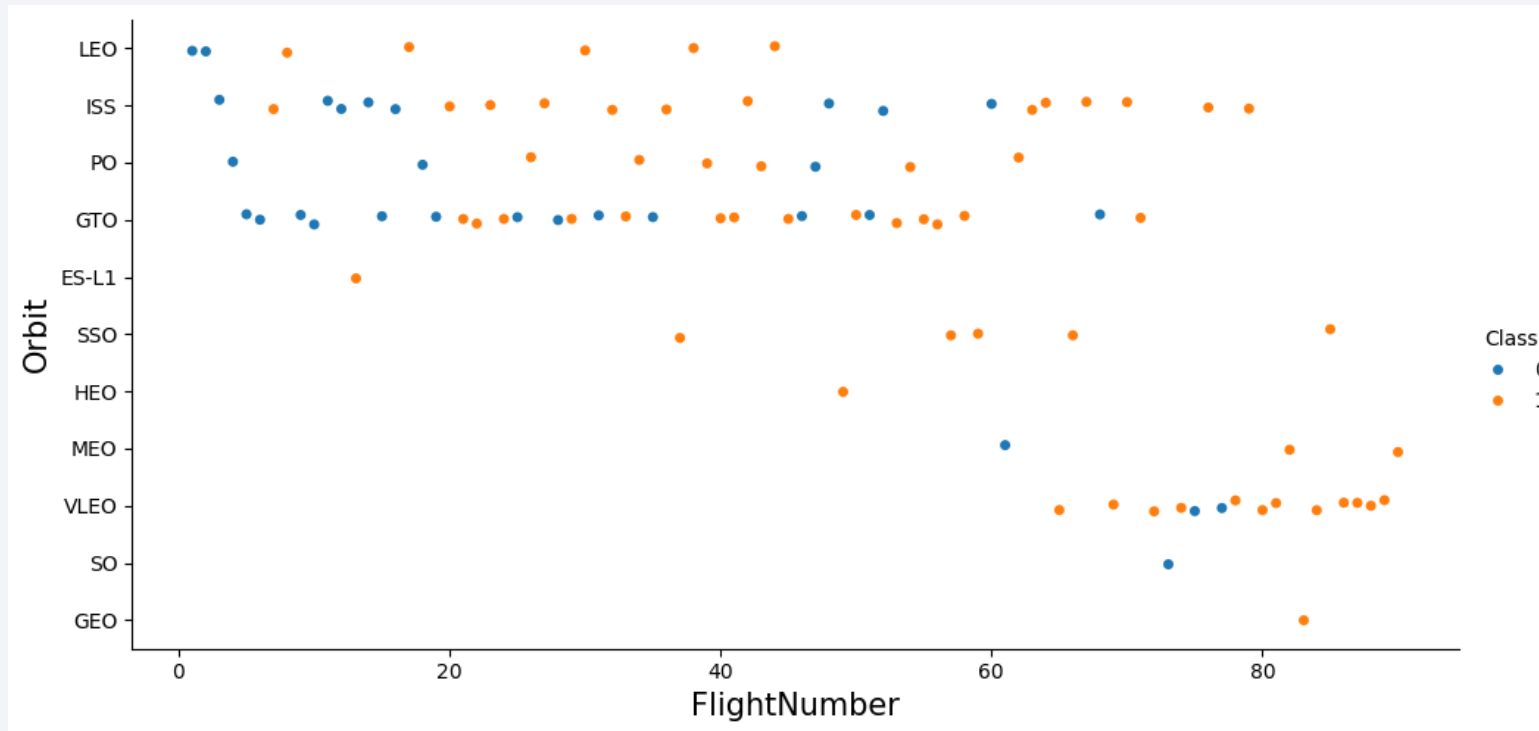
- We see that both the flight number increases and the payload mass are relevant features to the success of the mission.

Success Rate vs. Orbit Type

- We can see that ES-L1, GEO, HEO, SSO orbits have a 100% success rate.
- Whilst the SO type have a 0% percent.
- The rest have more expected rates.

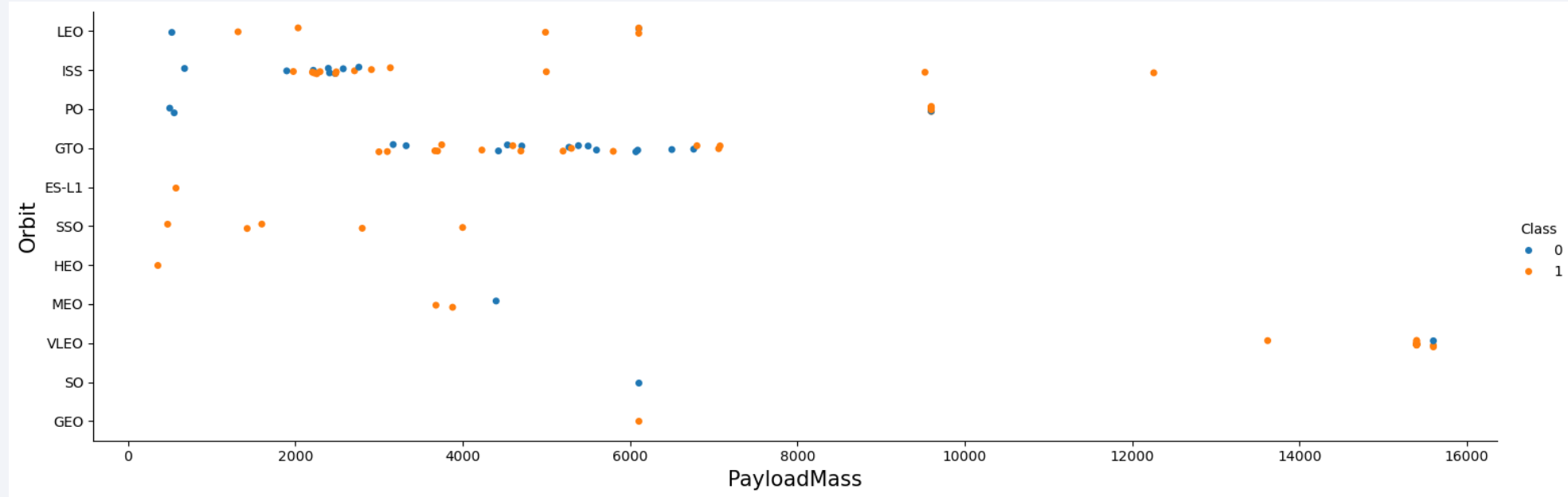


Flight Number vs. Orbit Type



- In the LEO orbit the success seems to be related to the number of flights.
- Looks like the MEO and VLEO orbits are reserved for rockets with a high number of flights

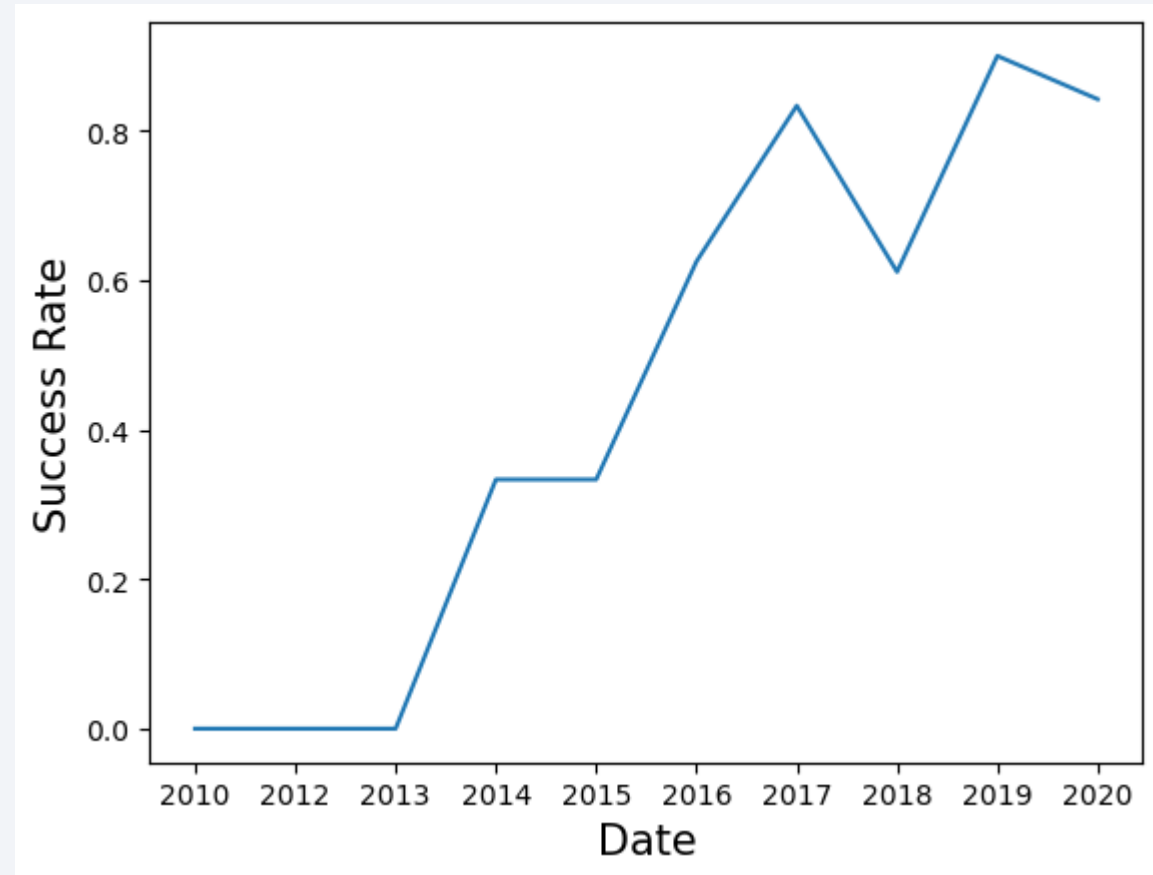
Payload vs. Orbit Type



- In LEO, ISS orbits higher payload seems to increase the success rate.

Launch Success Yearly Trend

- The success rate was 0 until 2013
- Overall, the success rate goes up from 2014 to 2020, with a dip on the year 2018.



All Launch Site Names

```
%sql SELECT DISTINCT Launch_Site FROM SPACEXTABLE
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

- There are 4 different launch sites

Launch Site Names Begin with 'CCA'

```
%sql SELECT * FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5
```

```
* sqlite:///my_data1.db
```

Done.

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

```
%sql SELECT SUM(PAYLOAD_MASS_KG_) FROM SPACEXTABLE WHERE Customer LIKE 'NASA (CRS)%'
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
SUM(PAYLOAD_MASS_KG_)
```

```
48213
```

Average Payload Mass by F9 v1.1

```
%sql SELECT AVG(PAYLOAD_MASS_KG_) FROM SPACEXTABLE WHERE Booster_Version LIKE 'F9 v1.1%'
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
AVG(PAYLOAD_MASS_KG_)
```

```
2534.6666666666665
```

First Successful Ground Landing Date

```
%sql SELECT MIN(Date) FROM SPACEXTABLE WHERE Landing_Outcome LIKE 'Success (ground pad)%'
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
MIN(Date)
```

```
2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

```
%sql SELECT DISTINCT Booster_Version FROM SPACEXTABLE WHERE Landing_Outcome LIKE 'Success (drone ship)%' AND PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

```
%sql SELECT Mission_Outcome, COUNT(Mission_Outcome) FROM SPACEXTABLE GROUP BY Mission_Outcome
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Mission_Outcome	COUNT(Mission_Outcome)
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

```
%sql SELECT DISTINCT Booster_Version FROM SPACEXTABLE WHERE PAYLOAD_MASS__KG_ = (SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTABLE)
```

```
* sqlite:///my_data1.db
```

Done.

Booster_Version

F9 B5 B1048.4

F9 B5 B1049.4

F9 B5 B1051.3

F9 B5 B1056.4

F9 B5 B1048.5

F9 B5 B1051.4

F9 B5 B1049.5

F9 B5 B1060.2

F9 B5 B1058.3

F9 B5 B1051.6

F9 B5 B1060.3

F9 B5 B1049.7

2015 Launch Records

```
%sql SELECT DISTINCT SUBSTR(Date, 6,2) AS Month_name, Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTABLE WHERE SUBSTR(Date, 0,5)='2015' AND Landing_Outcome LIKE 'Failure (drone ship)'
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Month_name	Landing_Outcome	Booster_Version	Launch_Site
01	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
04	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

```
%sql SELECT Landing_Outcome, COUNT(Landing_Outcome) FROM SPACEXTABLE WHERE Date BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY Landing_Outcome ORDER BY COUNT(Landing_Outcome) DESC
```

```
* sqlite:///my_data1.db
```

```
Done.
```

Landing_Outcome	COUNT(Landing_Outcome)
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The background is a deep blue gradient.

Section 3

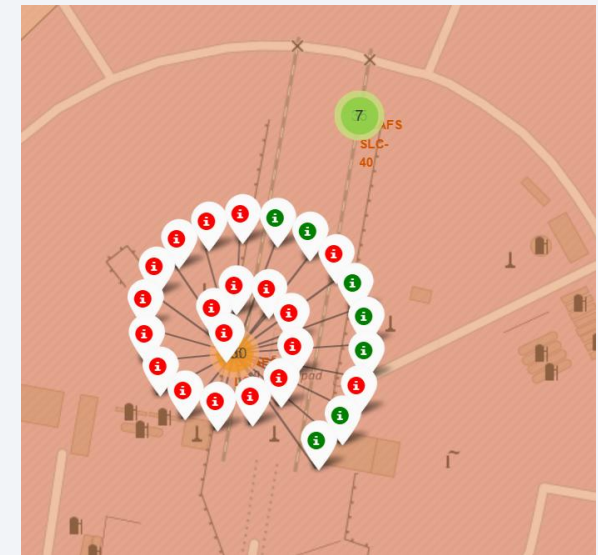
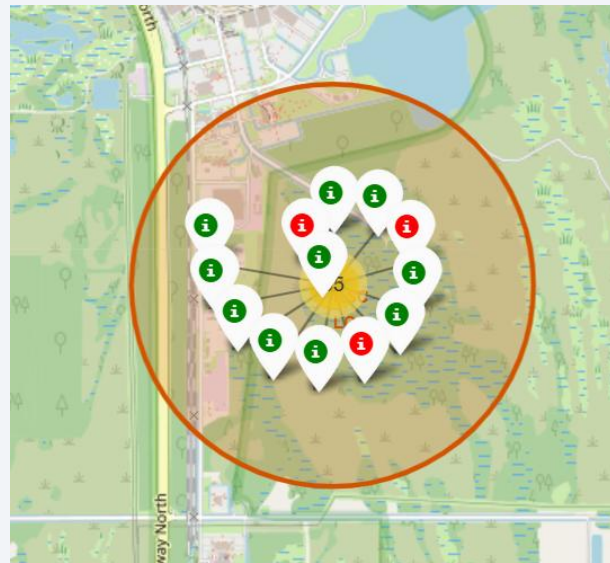
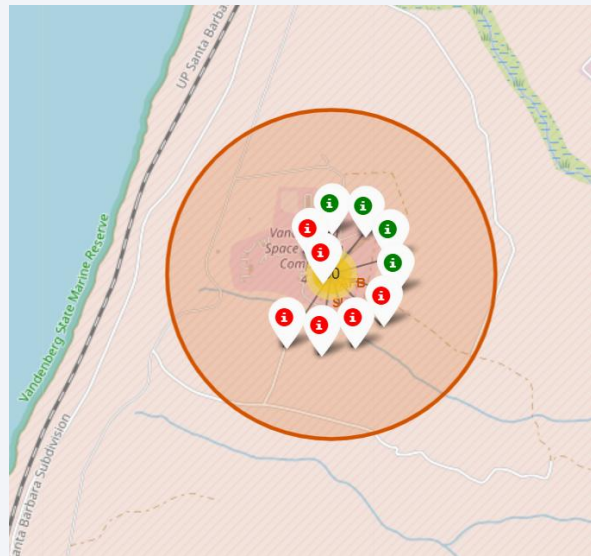
Launch Sites Proximities Analysis

Launch Sites on map



- All sites are located on a southern US coast.

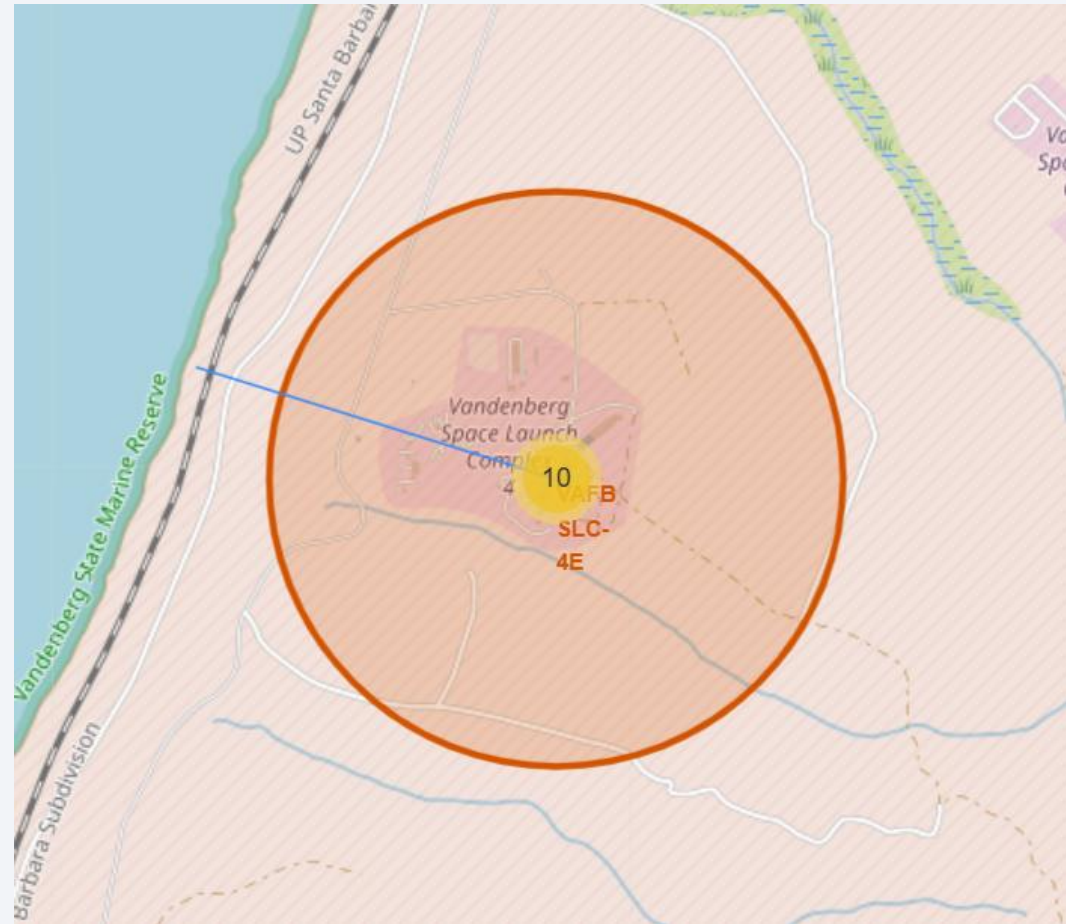
Launch outcome markers per site

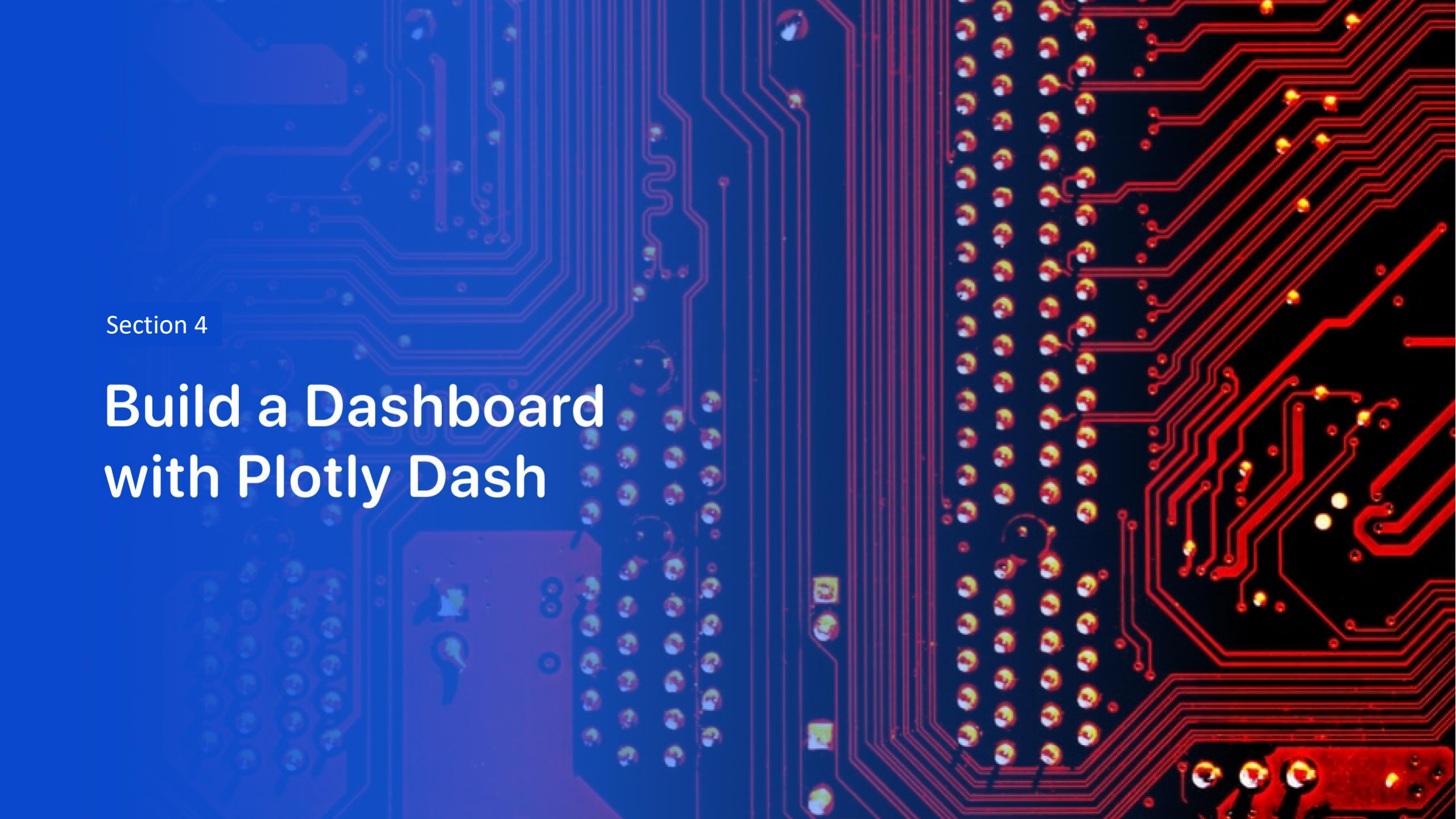


- The green markers indicate successful landings and red ones failed ones.

Distances between a launch site to proximities

- The launch VAFB SLC-4E site is located 1.30 km away from the west coast.

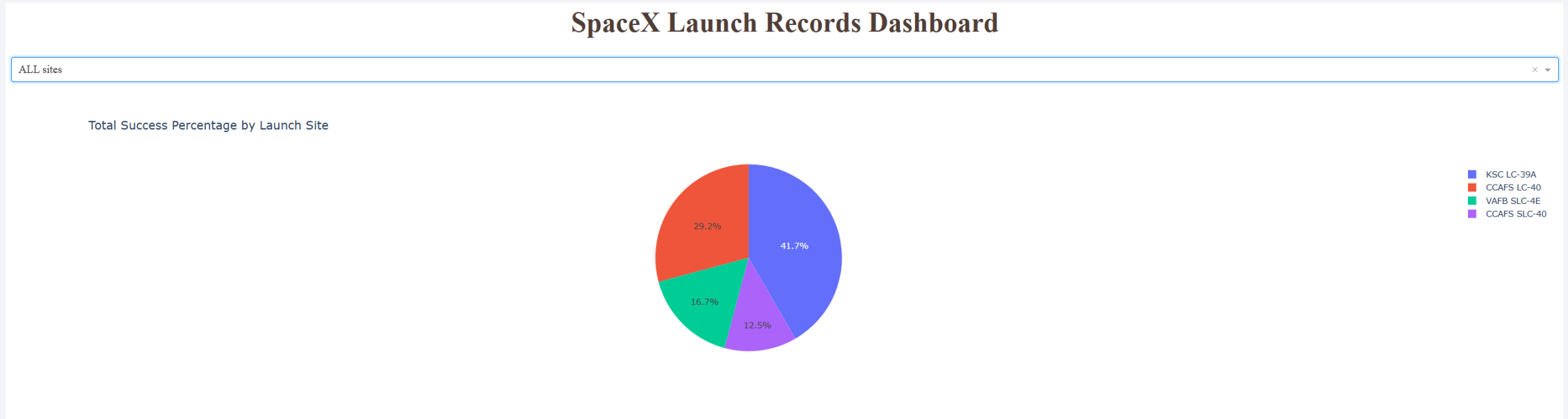




Section 4

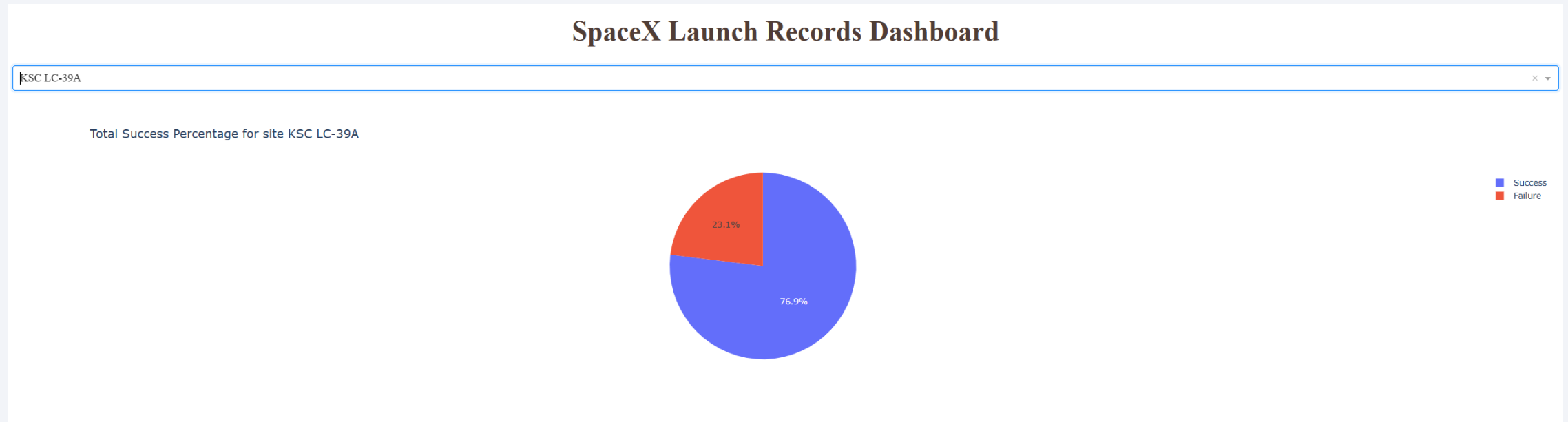
Build a Dashboard with Plotly Dash

Total Success Percentage by Launch site



- The most successful launch site is KSC LC-39A, followed by CCAFS LC-40.

Success percentage for KSC LC-39A



- The site has a success rate of 76.5% with 10 successful launches against 3 failures.

Payload vs Launch outcome scatter plot for all sites



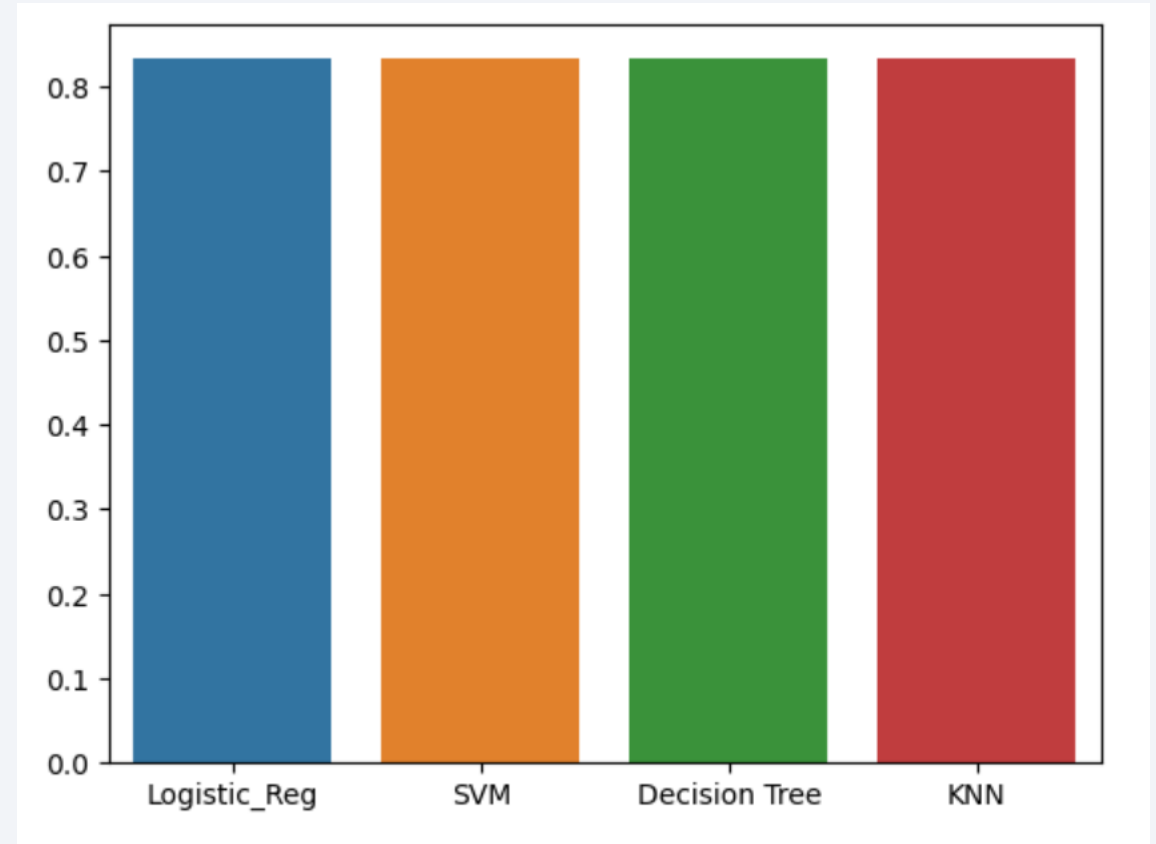
- The payload range with the largest success rate is between 3000 and 4000Kg.
- The most successful booster version is the FT.

Section 5

Predictive Analysis (Classification)

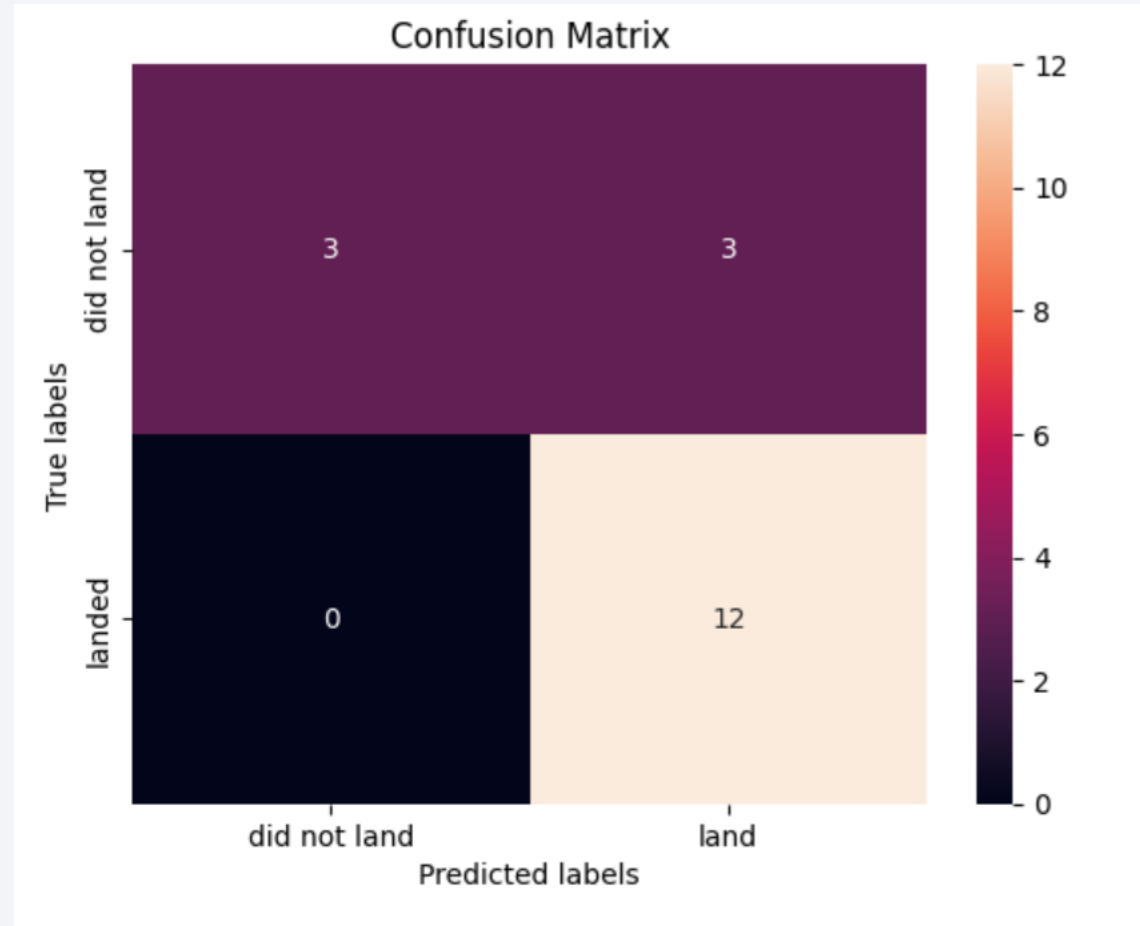
Classification Accuracy

- The results obtained on the test set for all models is around 0.8333
- There is no significant difference between models, so taking into account the accuracy results obtained during the training, we could say that the best performing model is the Decision Tree with a 0.873 accuracy during training.



Confusion Matrix

- Of the 18 cases in the test set, 15 were correctly predicted:
 - 3 did not land
 - 12 land
- And 3 were misclassified as land although they did not.



Conclusions

- After evaluating the prediction models, we observe that either one of them would perform relatively well with an 83% accuracy on the test set. And taking into account the accuracy score on the training set, the Decision Tree model has a slight edge over the rest.
- A larger dataset is needed in order to create a more significant test set, to ensure which model is the best performing.

Thank you!

